On This Page >

Introduction to Services

Clan's services are a modular way to define and deploy services across machines.

This guide shows how to **instantiate** a **service**, explains how service definitions are structured and how to pick or create services from modules exposed by flakes.

A similar term: **Multi-host-modules** was introduced previously in the nixus repository and represents a similar concept.

Overview

Services are used in `inventory.instances`, and assigned to *roles* and *machines* -- meaning you decide which machines run which part of the service.

For example:

```
inventory.instances = {
  borgbackup = {
    roles.client.machines."laptop" = {};
    roles.client.machines."workstation" = {};

    roles.server.machines."backup-box" = {};
  };
}
```

This says: "Run borgbackup as a *client* on my *laptop* and *workstation*, and as a *server* on *backup-box*". `client` and `server` are roles defined by the `borgbackup` service.

Module source specification

Each instance includes a reference to a **module specification** -- this is how Clan knows which service module to use and where it came from.

It is not required to specify the `module.input` parameter, in which case it defaults to the pre-provided services of clan-core. In a similar fashion, the `module.name` parameter can also be omitted, it will default to the name of the instance.

Example of instantiating a `borgbackup` service using `clan-core`:

```
inventory.instances = {

    borgbackup = { # <- Instance name

        # This can be partially/fully specified,
        # - If the instance name is not the name of the module
        # - If the input is not clan-core
        # module =  {
        #     name = "borgbackup";  # Name of the module (optional)
        #     input = "clan-core"; # The flake input where the service is
        # };

        # Participation of the machines is defined via roles
        roles.client.machines."machine-a" = {};
        roles.server.machines."backup-host" = {};
    };
}
```

Module Settings

Each role might expose configurable options. See clan's clanServices reference for all available options.

Settings can be set in per-machine or per-role. The latter is applied to all machines that are assigned to that role.

```
inventory.instances = {
    borgbackup = {
        # Settings for 'machine-a'
        roles.client.machines."machine-a" = {
```

```
            settings = {
                backupFolders = [
                    /home
                    /var
                ];
            };
        };

        # Settings for all machines of the role "server"
        roles.server.settings = {
            directory = "/var/lib/borgbackup";
        };
    };
}
```

Tags

Tags can be used to assign multiple machines to a role at once. It can be thought of as a grouping mechanism.

For example, using the `all` tag for services that you want to be configured on all your machines is a common pattern.

The following example could be used to backup all your machines to a common backup server

```
inventory.instances = {
    borgbackup = {
        # "All" machines are assigned to the borgbackup 'client' role
        roles.client.tags =  [ "all" ];

        # But only one specific machine (backup-host) is assigned to the
        roles.server.machines."backup-host" = {};
    };
}
```

Sharing additional Nix configuration

Sometimes you need to add custom NixOS configuration alongside your clan services. The `extraModules` option allows you to include additional NixOS configuration that is applied for

every machine assigned to that role.

There are multiple valid syntaxes for specifying modules:

```
inventory.instances = {
    borgbackup = {
        roles.client = {
            # Direct module reference
            extraModules = [ ../nixosModules/borgbackup.nix ];

            # Or using self (needs to be json serializable)
            # See next example, for a workaround.
            extraModules = [ self.nixosModules.borgbackup ];

            # Or inline module definition, (needs to be json compatible)
            extraModules = [
              {
                # Your module configuration here
                # ...
                #
                # If the module needs to contain non-serializable expres
                imports = [ ./path/to/non-serializable.nix ];
              }
            ];
        };
    };
}
```

Picking a clanService

You can use services exposed by Clan's core module library, `clan-core`.

🔗 See: List of Available Services in clan-core

Defining Your Own Service

You can also author your own `clanService` modules.

🔗 Learn how to write your own service: Authoring a service

You might expose your service module from your flake – this makes it easy for other people to also use your module in their clan.

💡 Tips for Working with clanServices

- You can add multiple inputs to your flake (`clan-core`, `your-org-modules`, etc.) to mix and match services.
- Each service instance is isolated by its key in `inventory.instances`, allowing to deploy multiple versions or roles of the same service type.
- Roles can target different machines or be scoped dynamically.

What's Next?

- Learn about Service Exports → - Share configuration data between machines
- Author your own clanService →
- Migrate from clanModules →

← Previous
Auto-included Files

Next →
Authoring a 'clan.service' module

Need help? Reach out to our community.